DT081A — Signal and Image Processing:

Laboratory 1 Report

Yacoub Dawli yada2100@student.miun.se

2026-10-29

1 Introduction

This lab explored signal generation, Fourier analysis, sampling, image resampling, quantization, quality measurements and noise reduction. Audio analysis also demonstrated aliasing.

2 Experimental Setup

I used Python, NumPy, SciPy, Matplotlib, Pillow and scikit-image, following the teacher's Lab 1 examples. Inputs were Lenna, four course TIFF images, and piano, rain and speech recordings. Noise used seed 2026.

3 Results and Analysis

3.1 Signal Generation and Visualization

The signals were a 10 Hz sine, 5 Hz square and 5 Hz sawtooth. The sawtooth $2((5t) \bmod 1) - 1$ makes rising ramps from -1 towards $+1$. Changing the sine to 20 Hz doubled its cycle count. The sign and ceil expressions differed only at $t = 0$: 0 versus -1. Ceil did not extend the positive part. At an exact sine value of -1, ceil would instead give -3. Adding the sine and square gave jumps and curves, a range near -2 to $+2$, and a 0.2 s period (Figure 5). Synthesizers use waveform shape to change sound.

3.2 Fourier Transform and Frequency Analysis

The 10 and 20 Hz components had amplitudes 1 and 0.5. Their FFT peaks were at 9.990 and 19.980 Hz. Adding 30 Hz at amplitude 0.25 gave a smaller 29.970 Hz peak. The endpoint-including grid has spacing 1/999 s, explaining the offsets. Noise of standard deviation 0.2 raised the spectral background, while the main peaks remained visible (Figure 1).

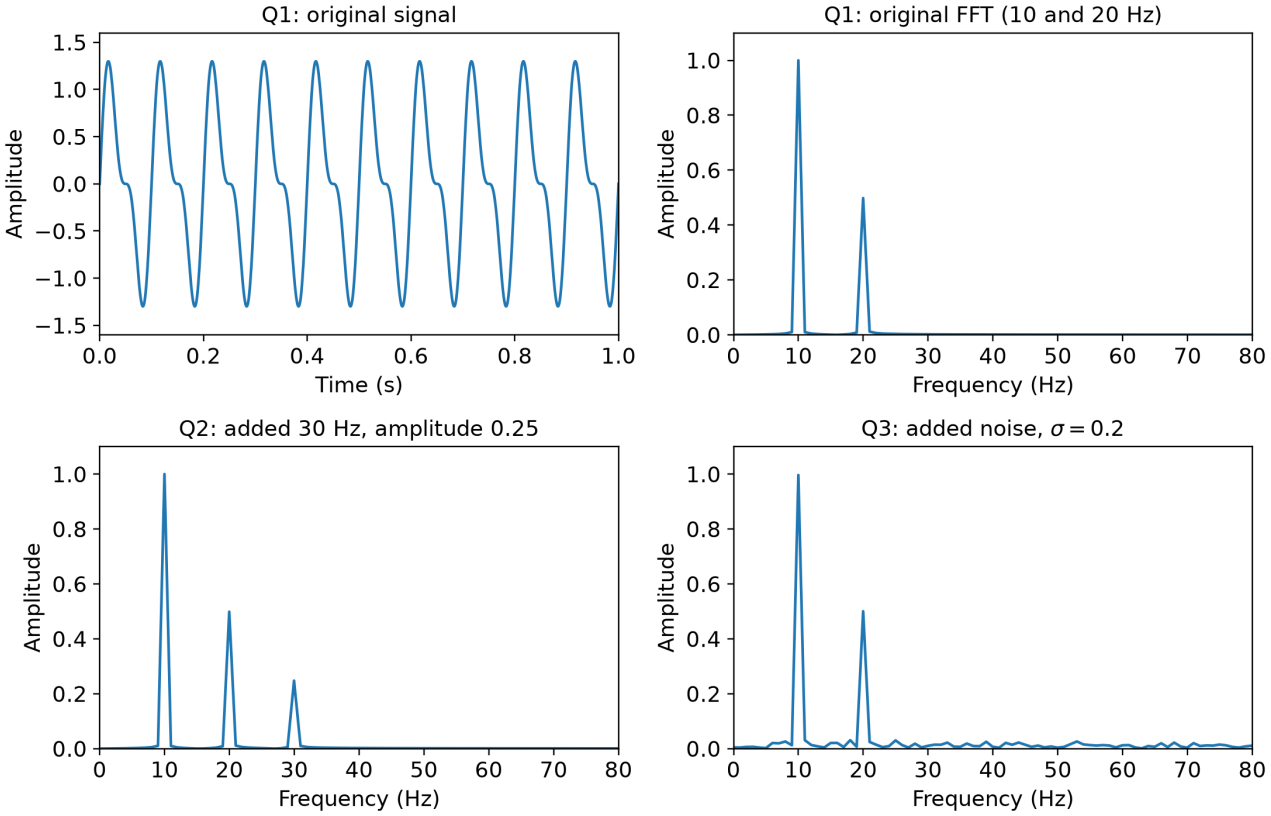


Figure 1: Q1–Q3: original signal and FFT (top), FFT after adding 30 Hz and FFT after adding noise (bottom). All FFT panels use the same amplitude scale.

The chirp phase was $2\pi(10t + 20t^2)$, giving $f(t) = 10 + 40t$: 10–50 Hz in one second. Its cycles became closer together, and its whole-signal FFT spread across a frequency range. Figure 2 shows the closer cycles in time and the broad FFT spectrum; a whole-signal FFT does not show when each frequency occurs.

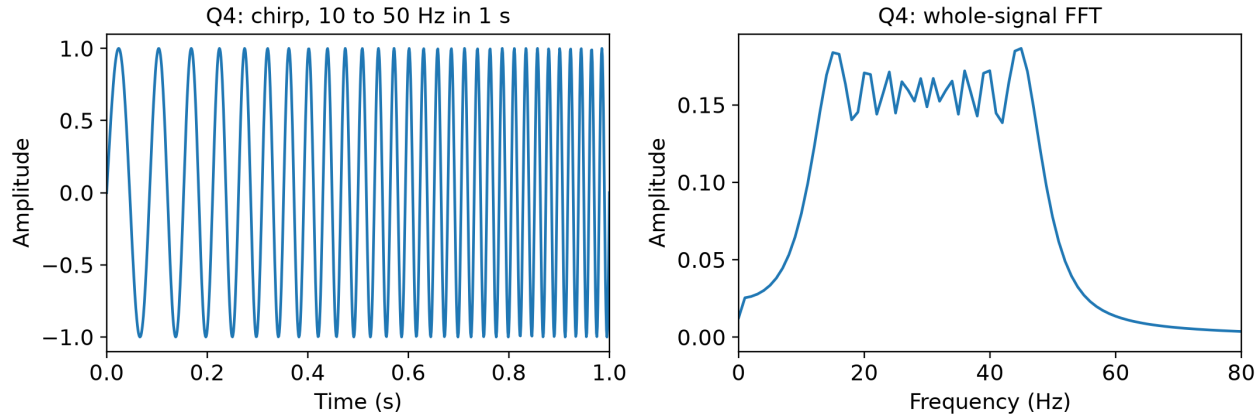


Figure 2: Q4: chirp waveform and whole-signal FFT amplitude, using the same signal and FFT scaling as the implementation.

Audio equalizers use frequency analysis to change selected bands.

3.3 Sampling and Aliasing

The Nyquist rates are 20 Hz for a 10 Hz signal and 40 Hz for 20 Hz. At 15 Hz sampling, 10 Hz folded to a 5 Hz alias; at 25 Hz it did not fold. Sampling 20 Hz at 30 Hz gave a 10 Hz alias. Signed matches −5, −10 and −1 Hz describe reversed phase progression. The 10 Hz/11 Hz case explains slow or reverse wagon-wheel motion (Figure 3). In photography, undersampled repeating detail can create false moire patterns.

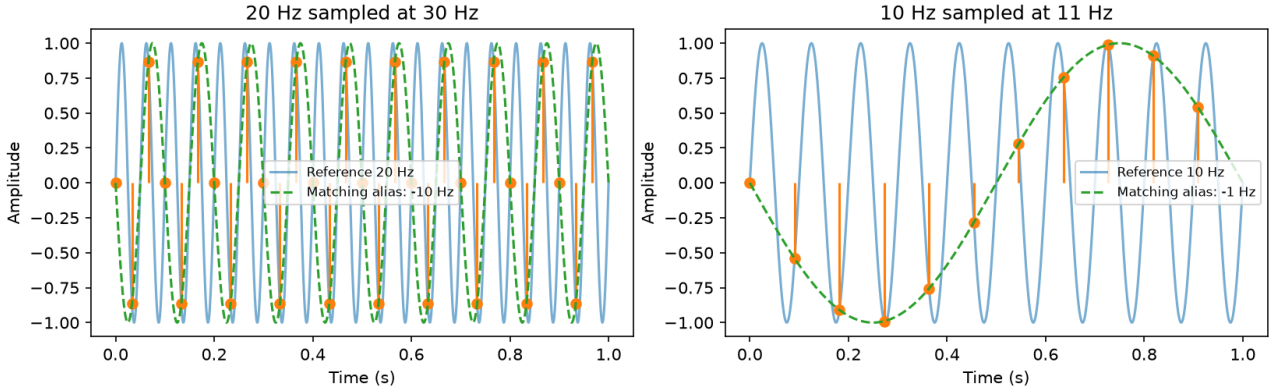


Figure 3: Original waves, orange samples and dashed matching aliases. The 10 Hz/11 Hz example appears to move slowly backwards.

3.4 Image Resampling and Requantization

The main Lenna grayscale image had 512 × 512 pixels. Simple resampling kept every nth pixel and repeated it to full size. Factor 2 changed fine edges; factor 4 showed clear blocks, and factor 8 was worse. Requantization reduced grey levels, causing bands rather than blocks (Figure 4). At this size, 6 bpp looked close to the original; 4 bpp showed clear bands and 2 bpp strong bands. These allow 64, 16 and 4 levels respectively. Visibility depends on the image and viewer.

The combined operation resampled and then requantized. Factor 2 with 6 bpp was my best balance; factor 4 reduced more but looked blockier (Table 1). For factor n and bit depth b, the ideal ratio is $8n^2/b$, assuming packed reduced pixels from an 8 bpp original and excluding headers. The expanded NumPy arrays are not compressed files.

For a fair comparison, both simple and bicubic methods reduced each dimension by four and reconstructed to 512 × 512. Pillow bicubic resizing looked smoother, although fine detail softened. Its higher PSNR agrees with this result. One run took 1.504 ms for simple resampling and 4.181 ms for bicubic. Bicubic does more calculation; timings vary.

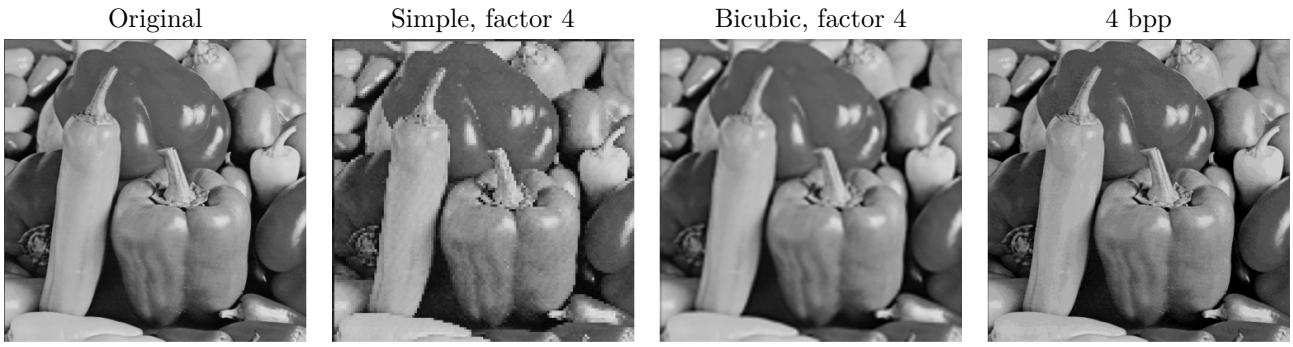


Figure 4: Peppers example, all 0–255. Bicubic smooths block edges; requantization creates bands. Table 1 gives the separate Lenna measurements.

Resampling lost portrait texture, grass detail and small aerial features. Bicubic reduced block edges but could not restore everything. Requantization kept much texture, while smooth areas showed bands, especially on peppers.

3.5 PSNR and SSIM

PSNR used $10 \log_{10}(255^2/\text{MSE})$, with floating-point subtraction before squaring. Table 1 shows more pixel error after resampling than after requantization. The requantized image also looked closer to the original, so the PSNR ranking agreed with this comparison.

Table 1: Image quality and selected ideal reductions. A dash means not listed.

Processing	Ideal reduction	PSNR (dB)	SSIM
Simple resampling, factor 4	—	22.71	0.6589
Requantization, 4 bpp	—	29.21	0.8832
Factor 2, 6 bpp	5.33×	28.14	—
Factor 4, 6 bpp	21.33×	22.66	—
Bicubic, factor 4	—	28.84	—

The parameter sweep covered factors 1, 2, 4 and 8 at 1–8 bpp. Higher bit depths improved quality, but resampled images levelled off because spatial detail was already lost. At 8 bpp, factors 2, 4 and 8 gave 28.30, 22.71 and 19.23 dB. The unchanged image had infinite PSNR. Figure 6 shows the sweep.

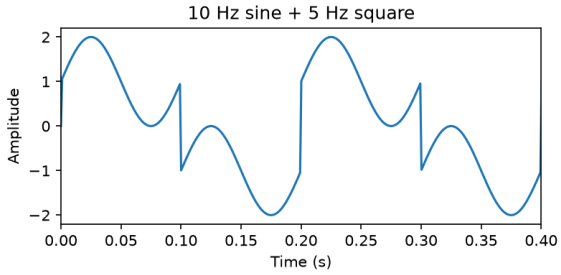


Figure 5: Composite waveform: 10 Hz sine plus 5 Hz square.

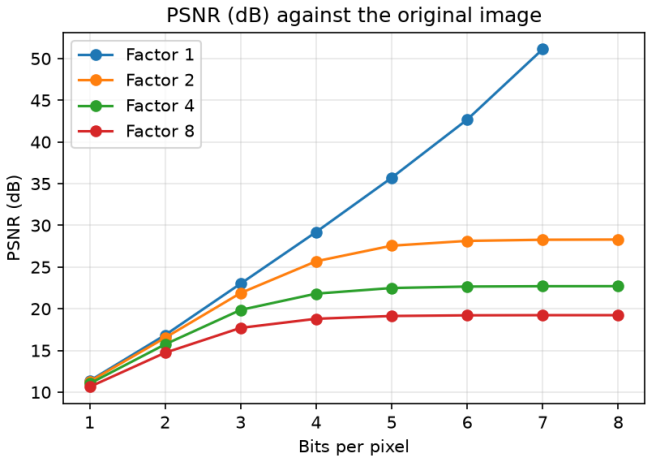


Figure 6: Lenna PSNR versus bit depth for the four resampling factors.

SSIM compares local brightness, contrast and structure, while PSNR measures average squared pixel error. SSIM used scikit-image's default 7×7 window with `data_range=255`. SSIM also favoured requantization here. Neither metric guarantees what every viewer will prefer.

3.6 Noise Addition and Reduction

Gaussian noise was added with seed 2026, clipped to 0–255 and converted to 8-bit pixels. Slight grain appeared in smooth areas at standard deviation $\sigma_n = 5$, giving 34.11 dB. This lies within the quoted 30–50 dB quality range, showing that noise can still be visible within that range. Visibility is subjective, rather than a fixed PSNR boundary.

At $\sigma_n = 150$, PSNR was 8.46 dB and the main shape remained visible. At $\sigma_n = 500$, PSNR was 6.24 dB and the content was difficult to recognize, although faint shapes could remain. Heavy clipping means the added noise's standard deviation is not the final pixel-error standard deviation.

Gaussian filtering with spatial $\sigma = 1$ improved the same $\sigma_n = 20$ noisy image from 22.13 to 30.08 dB against the clean original (Figure 7). Grain decreased, but texture and edges softened. Floating-point pixels avoided rounding during filtering.

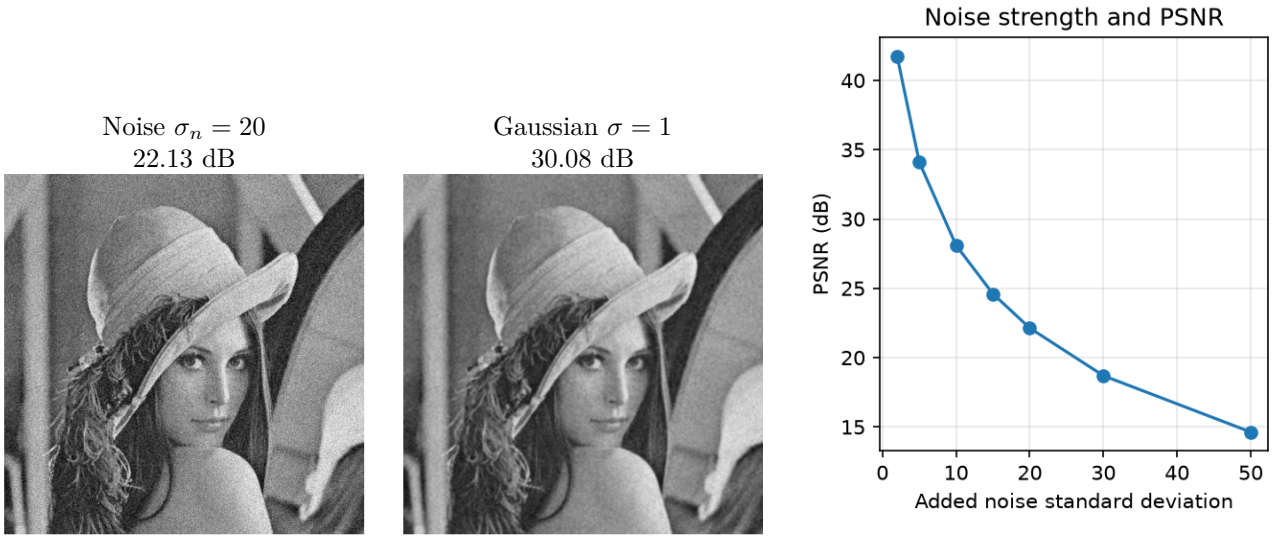


Figure 7: Same noisy Lenna before/after filtering, and PSNR versus added noise strength. All PSNR values use the clean original; image limits are 0–255.

A more advanced method, Non-Local Means, averages similar image patches. It can preserve texture and edges better than nearby averaging, but needs more calculation and suitable settings (scikit-image guide).

3.7 Audio Signal Analysis

I loaded a piano note, rain and speech. Stereo channels were averaged in floating point. Separate normalization means the plots do not compare original loudness.

Table 2: Audio measurements. The first three rows use complete clips.

Clip	Sampling rate (Hz)	Duration (s)	Strongest FFT peak (Hz)	Highest bin at 1% (Hz)
Piano	44100	0.5018	219.21	4832.69
Rain	22050	30.0301	272.56	11021.15
Speech	44100	1.3013	290.48	15737.16
First 0.5 s speech	44100	0.5000	290.00	—

The highest-bin rule counts amplitudes at least 1% of the strongest non-zero frequency peak. Weaker components can extend higher, so this is not an exact physical limit. The strongest peak is a separate measurement.

Piano showed fading regular oscillations, spectral peaks and harmonic bands. Rain was less periodic with broad frequency content. Speech had bursts, gaps and changing bands. Brighter regions mean greater power at that time and frequency.

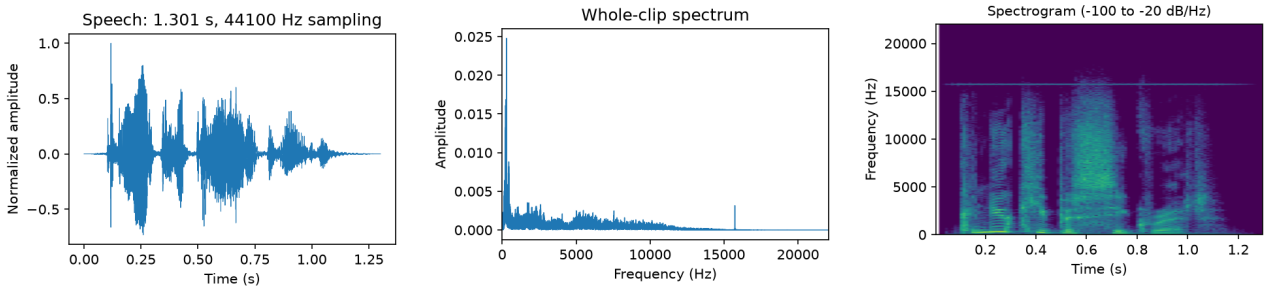


Figure 8: Speech waveform, whole-clip FFT amplitude and spectrogram. Spectrogram: 1024-sample windows, 512 overlap, normalized amplitude, −100 to −20 dB/Hz.

Cropping speech from 1.3013 s to 0.5 s kept 22050 samples. FFT spacing changed from about 0.768 to 2 Hz. The main peak stayed near 290 Hz, but other strengths changed because only the beginning was included. Both spectra used the same scale.

Downsampling reduced the number of audio samples and lowered the highest frequency that could be represented correctly. Without an anti-aliasing filter, higher frequencies could appear as false lower frequencies, potentially reducing sound quality. For the 2500 Hz test tone, the frequency remained unchanged at a 6000 Hz sampling rate but appeared at 1500 Hz when sampled at 4000 Hz and 500 Hz when sampled at 2000 Hz. This demonstrates that aliasing occurs when the sampling rate falls below twice the signal frequency, which is 5000 Hz in this example.

Spectral noise reduction could estimate noise magnitudes from a quiet part, reduce FFT magnitudes while keeping phases, and use an inverse FFT to return to time samples. It may also damage quiet wanted sounds.

4 Discussion

Sampling chooses time points for signals and audio, or pixel positions for images. Quantization limits amplitude or grey levels. Too few samples can cause aliasing; too few levels introduce rounding errors.

Images and audio share sampling, Fourier analysis and filtering, but images have two spatial directions and mono audio one time direction. Communication systems need suitable sampling, medical images need detail with controlled noise, and multimedia balances data size against picture and sound quality.

5 Conclusion

The lab showed how sampling, quantization and noise change represented information. FFT analysis exposed frequency content and aliasing. Quality measurements helped compare results, while visual observations showed their limits.

Appendix: Code Snippets

A. Chirp generation. The sweep duration sets how quickly frequency changes.

```
def make_chirp(time, start_frequency, end_frequency, duration):
    slope = (end_frequency - start_frequency) / duration
    phase = 2 * np.pi * (start_frequency * time + 0.5 * slope * time**2)
    return np.sin(phase)
```

B. PSNR calculation. Converting to float before subtraction prevents 8-bit wraparound.

```
    error = clean.astype(float) - processed.astype(float)
    mse = np.mean(error**2)
    if mse == 0:
        return float("inf")
    return 10 * np.log10(255**2 / mse)
```